Instructor Notes

Pedro Pathing 3 for FIRST Tech Challenge · Deck A: Concepts, now starting with the PathLab demo

Companion to **FTC_PedroPathing_DeckA_Concepts_v4.pdf** (25 slides)

The lesson in one line. Students **watch** the robot drive a path on the real field, **write** what they saw and wonder, **discuss**, and only then **learn the ideas**. Every concept slide ends with a “Back to the demo” strip that points at something they just saw. At the end, they answer their own questions.

Timing. About 2 hours in three sessions: slides 1–8 (about 44 min, including the demo), slides 9–16 (about 39 min), slides 17–25 (about 44 min; there is a natural break after slide 21). Each entry below gives a suggested time, what to say, a question to ask, and what to watch out for. Answers are in [brackets].

If a student asks something these notes do not cover, “I don't know, let's look it up” is a good answer. Everything on the slides is sourced from the Pedro docs, the goBILDA Pinpoint guide, or the PathLab project documents. The footer of each slide says which.

What I could not check. I read the PathLab user guide, design notes (HLD) and code walkthrough (LLD). I have **not** read the PathLab source code, and I have not seen the robot run. The design notes say the code compiled but had not yet been run on the robot, and flag two open risks (see the table below). **Do a full dry run before class** and keep a short screen recording as a backup.

The demo: run sheet

When	What to do	Source
Before class	Deploy the code. Connect the laptop to the robot Wi-Fi and open FTC Dashboard at `http://192.168.43.1:8080/dash`. Clear about 4 × 3 tiles. Tape an X at the Starting Point (24, 24). Keep the default points (Dashboard → Configuration → Path Poses). Dashboard edits are not saved on the robot.	PathLab guide, LLD
Each test	Choose the test on the Driver Station. Place the robot's centre on the X, facing 0° (right). Press INIT: the planned path shows on the Dashboard field. Press PLAY, then **A** to run. B stops the robot at any time. Y sends it back to the start.	PathLab guide, LLD
Test 1 then Test 3	Test 1: straight line, keeps heading (constant). Test 3: curve with one control point, faces the way it travels (tangent). Both go from (24, 24) towards the End point (72, 60), pause 2 seconds, then drive back.	PathLab guide, LLD
Slide 3	Students write for about 5 minutes. Then run the discussion (next section).	This deck

If this happens	Likely cause	What to do
Test is not on the Driver Station	Code not deployed	Build and deploy from Android Studio.
Dashboard does not load	Laptop not on robot Wi-Fi	Connect and reload.
Pressing A does nothing	Not in the READY step, or the Starting Point was moved	Read the Message line on the Driver Station; press Y if it asks.
Robot seems stuck while driving	Possibly `isBusy()` staying true (a risk flagged in the design notes), or an unreachable End point	Press B. Check the End point. Say honestly: “we found something to look into.”
Robot faces the wrong way at the start	Wrong heading or placement	0° is right, 90° is up. Re-place the robot.
Picture on the Dashboard flickers	Flagged in the notes as possible and unverified	Carry on. Do not try to fix it in class.

The discussion (slide 3, about 8 minutes)

> **Do not teach yet.** Draw three columns on the board: **We saw**, **We wonder**, **We guess**. Ask each student for one item from their notebook and write it down without correcting. If a student guesses right, say “interesting, let's keep that and check it later.” Keep the board visible: slide 23 returns to it. Push for specifics: “Where exactly did it start? Which way was the front pointing?” If the class misses it, point out that Test 1 and Test 3 reach the same end spot but the robot is facing differently when it gets there.

Part 0 · The demo

1 Title slide

1 min.

Say: Today we start with the robot. Watch first, think second.

Back to the demo: Show the title, then go straight to slide 2.

---

2 Today's demo: a robot drives a path

8 min. Run Test 1, then Test 3, with this slide on screen.

Say: Read the three lines: a robot drives from a start spot to an end spot, first a straight line and then a curve, then it waits 2 seconds and drives back. Then run the tests. Say nothing about how it works yet.

Ask the class: Before the first run: what do you expect the robot to do? [Any answer.]

Watch out: If the first run goes wrong, do not hide it. “Let's see what happened” is a good start to a discussion about how robots are tested.

---

3 While you watch, write in your notebook

About 13 min: 5 writing, 8 discussion.

Say: Give them the three headings: I saw, I wonder, My guess. Walk around while they write. Then run the board discussion above.

Ask the class: What is different between the line and the curve? [Shape of the route; and the way the robot was facing along it and at the end.]

Watch out: Do not let this turn into a quiz. The aim is a list of questions that the rest of the deck will answer.

---

4 Here is what the robot did

4 min.

Say: Go through the table. Start (24, 24) facing 0° (right). End point (72, 60). Test 1 is a straight line of about 60 inches that keeps facing 0°. Test 3 bends towards Control Point 1 at (60, 24), is about 65 inches long, and ends facing about 72°. Then it waits 2 seconds and drives back.

Ask the class: Check your notes. What did you see that is not on this list? [Often: how smooth it was, whether it slowed down, how close it landed.]

Watch out: These numbers are the default points and the figures in the PathLab user guide. If you edited the points on the Dashboard, change the numbers you say. The “about 65 in” and “about 72°” are from the guide, so measure your own robot if a student asks. The End point also has a heading of 90° in the table. The guide says only Test 2 uses it, which is why Test 1 stayed at 0°. Hold that for slide 10.

Part A · Why we need this

5 Why “drive for 2 seconds” fails

4 min.

Say: Each row is a different way timing goes wrong. The footer says this is general robotics reasoning and not a claim from the Pedro docs. That honesty is worth saying aloud.

Back to the demo: The demo robot drove out, waited and came back. Could timing do that to the same spots, run after run? [No: it cannot notice or correct a difference.]

Ask the class: You set 2 seconds at 50% power and it worked at lunch. Why might it be worse at the end of the day? [The battery is lower, so the motors turn slower.]

Watch out: Do not give numbers such as “10% slower”. If asked how much, say “I don't know. We would have to measure it.”

---

6 Arrive at the same spot, every match

3 min.

Say: Three needs: know where I am, know the route, keep correcting. The bottom box is the one Pedro claim on this slide: its docs say it follows paths while correcting for disturbances and plans its braking from measurements of the robot.

Back to the demo: Did the robot come back to the taped X? How could we check? [Measure from the tape.] If nobody measured, say “we would have to measure it”; do not guess.

Ask the class: Which of the three do you think is hardest? [No single answer. Let them argue.]

Watch out: Do not promise “perfect” paths. Pedro corrects errors. It does not remove them.

---

7 Your phone's map app already does this

5 min. The key slide for the analogy.

Say: Go row by row, top to bottom. Students will come back to this table all lesson. “Map settings” is a little stretched, but it sets up PoseFactory later.

Back to the demo: On the Dashboard picture, find the planned path (the route) and the robot's position. Which is the plan and which is the reality? [Planned path = plan; robot position = reality.]

Ask the class: Which row matches “recalculating…”? [The follower correcting back onto the path after a push.]

Watch out: The analogy has limits. A phone picks your route. In Pedro, **we** design the path. Say so.

---

8 Pedro Pathing is a box

5 min. The big picture.

Say: Point at the three parts. **Inputs:** sensor readings from the Pinpoint (x, y and heading, again and again), the starting Pose you give it, and your path. **Inside:** the localizer answers “where am I?” and the follower answers “how do I get there?” (it brakes early using Foresight). **Output:** power for each drive motor. The robot moves, the Pinpoint measures again, and the loop repeats. A driver is only as good as the GPS: if the localizer is wrong, the follower steers confidently to the wrong place.

Back to the demo: The path you saw drawn on the Dashboard was an input. The wheels turning was the output. Ask: what happened inside the box? Let them guess; the next slides answer it.

Ask the class: The robot gets pushed sideways. Which input changes? [The sensor reading: the Pinpoint reports a new position.] Which input stays the same? [The path.] The robot drives smoothly but ends up in the wrong place. Which half is broken? [The localizer: it thinks it is somewhere it is not.]

Watch out: This picture is our own simplification, not a diagram from the Pedro docs. The docs' Introduction describes the pieces (localizer, follower, Foresight). Pedro has more settings than the box shows, so do not claim it is complete. Students often think Pedro is only for autonomous. Deck B shows it driving in TeleOp too.
Natural break point after this slide.

Part B · Where am I?

9 The field is a grid

4 min.

Say: Like Battleship: two numbers find a square. x is across, y is up, (0, 0) is the bottom-left corner. The field is 141.5 inches each way. The demo started at (24, 24) and ended near (72, 60): find both.

Ask the class: Roughly where is (72, 60)? [About halfway across and a bit under halfway up.] About how many tiles from the start to the End point, across and up? [48 in across is about 2 tiles; 36 in up is about 1½ tiles, with one tile about 23.6 in.]

Watch out: Pedro uses its own coordinates, and its docs call them nonstandard compared with the FTC SDK. Do not go into why. The Pedro docs' conversion example assumes a 144-inch field, but the Pedro Visualizer, the PoseFactory page and the PathLab notes all say 141.5. We use 141.5. If a student finds the 144 figure, “the docs differ, so we use the Visualizer's value” is the honest answer.

---

10 A Pose is a spot plus a facing

5 min.

Optional demo (skip if not possible): Walk to a spot (x and y), then turn on the spot (heading). Same position, different Pose.

Say: A Pose has three numbers: x, y and heading. Heading 0° is facing right, 90° is up, 180° is left, 270° is down. Turning counter-clockwise makes the heading go up. The demo robot started at (24, 24) facing 0°.

Back to the demo: The End point on the Dashboard has heading 90°, but in Test 1 the robot stayed facing 0°. Hold that thought: the answer is on slide 19. [The PathLab guide says only Test 2 uses the End point's heading; Test 1 keeps the starting heading.]

Ask the class: I stand in the corner facing the door, then turn to face the window. Did my x and y change? [No.] Did my Pose change? [Yes: the heading.]

Watch out: Heading is in **degrees** on this slide. Pedro's own readings come back in radians (slide 12). Say the units warning twice.

---

11 Three ways to turn. One is the heading

8 min. Optional: open our interactive “Yaw, Pitch & Roll”.

Optional demo (skip if not possible): Everyone shakes, nods and tilts their head. Or, if you have a screen, open the interactive and drag the sliders; orbit the camera to show that the numbers do not change when only the viewpoint changes.

Say: Each turn has an **axis of rotation**: the imaginary line the robot spins around, like a pole through a spinning top. **Yaw**: the **Z axis** points straight up and down (a turntable, or shaking your head “no”). **Pitch**: the **Y axis** runs side to side (a seesaw, or nodding “yes”). **Roll**: the **X axis** runs front to back (a rolling log, or tilting your head). On a flat field only yaw matters, and yaw is the robot's heading.

Then the IMU, in three sentences. It stands for Inertial Measurement Unit. A gyroscope inside feels how fast it is turning. The computer adds up all the little turns to get the heading. The Pinpoint has one built in (goBILDA guide).

Back to the demo: The heading numbers in the demo (0°, 90°) are yaw, in degrees.

Ask the class: Which axis does a spinning office chair turn around? [Straight up and down: yaw.] What changes if the robot drives up a ramp? [Pitch.] If it starts to tip over sideways? [Roll.] Which one is the robot's heading? [Yaw.]

Watch out: The gyroscope and adding-up explanation is general knowledge, not from the goBILDA guide. The guide says only that the Pinpoint has an internal IMU that tracks heading, and that it calibrates at power-up (about 0.25 seconds, robot stationary). goBILDA's product name says “IMU sensor fusion for 2 wheel odometry”, so the Pinpoint combines the IMU with the two pods. I do not know the details of how; say so if asked. Do not go into gimbal lock or quaternions. The axis letters are the robot's own axes: x forward and y left (goBILDA guide), and z up (the usual third axis). The Pinpoint board even prints a Z with a rotation arrow. The pictures show example turns, not measured values. The guide says a counter-clockwise turn reads as positive heading, but another passage in the same guide seems to say otherwise and it does not resolve this. If a robot's heading reads the wrong way, check it on the robot.

---

12 A PoseFactory makes every Pose the same way

5 min. An important concept.

Say: Think of a stamp: set the rules once, and every Pose you stamp follows them. The order is always x, y, heading. The code on the slide uses the demo's Start and End point. Declare Poses once at the top of the class, not inside loop().

Back to the demo: On the Dashboard you type points in degrees. The PathLab notes say the code uses `PoseFactory.degrees()` for every pose. The notes warn that a bare `.constant(90)` would mean 90 *radians*, which is why heading is always passed as a Pose.

Ask the class: You type `p.of(24, 24, 90)`. Is 90 degrees or radians? [Degrees, because the factory was made with `degrees()`.] What does `follower.pose()` give back? [Heading in radians.]

Watch out: Do not go deep on radians. “Another unit for angles” is enough. Source: Pedro docs, Creating a Pose and TeleOp Usage.

---

13 Draw it for one alliance. Flip it for the other

4 min.

Say: `mirrorX(70.75)` flips every Pose across the line x = 70.75, the middle of the 141.5-inch field. Blue draws once, red comes for free.

Back to the demo: Try it on the demo's start. For the heading, say “we would check in the Visualizer.”

Ask the class: A blue robot starts at (30, 30). Where does the red mirror start? [(111.5, 30), because 141.5 − 30 = 111.5.] Now the demo's start at (24, 24)? [(117.5, 24).]

Watch out: The docs describe positions flipping. They do not say what happens to headings. Say that openly. “I don't know. We will test it in the Visualizer” is the right answer. Copy the code exactly as the docs show it. The PathLab demo itself does not need mirroring; this slide is a preview.

---

14 Odometry: add up every little movement

6 min.

Optional demo (skip if not possible): Two students walk heel to toe along tape with eyes closed. Compare where each stops.

Say: Three steps. (1) A small wheel rolls on the floor; it has no motor. (2) An encoder counts how far it turned. Distance = turns × the length around the wheel. For example, 5 turns of a wheel that is 100 mm around is 500 mm. (3) The computer adds up all the tiny moves, starting from the Pose you gave it. The total is where the robot is now. A forward pod, a sideways pod and the IMU together give a full Pose. The numbers on the slide are for goBILDA's 4-Bar pod: a 32 mm wheel (100.53 mm around) and 2,000 counts per turn, so one count is about 0.05 mm (100.53 ÷ 2,000).

Back to the demo: After the run, did the robot land exactly on the End point? The telemetry reports how far off the path it was. Off the path according to whom? [According to the robot's own measurement of its position. A tape measure on the floor is the independent check.] **Check first:** I have not read the PathLab code, so confirm that "biggest miss" is worked out from `follower.pose()` before you say so.

Ask the class: A wheel is 100 mm around and turns 3 times. How far did it roll? [300 mm.] The pod counts 2,000 per turn: how many counts is that? [6,000.] Why do small errors matter more the longer the robot drives? [Each position is built by adding up everything before it, so errors pile up. This is drift.]

Watch out: I do not know which pods our robot has. goBILDA also sells a Swingarm pod with a 48 mm wheel, and the guide lists a different ticks-per-mm for it. Check ours before quoting the 32 mm figures. The photo is goBILDA's product image; the pods in it look like the 4-Bar design but I cannot confirm that from the image. Do not say pods never slip; say only that they roll freely and do not drive the robot. "Distance = turns × length around the wheel" is general physics, not a Pedro or goBILDA claim.

---

15 Meet the goBILDA Pinpoint

4 min.

Say: This is what the real parts look like: two odometry pods and the Pinpoint computer. The pods are small wheels with no motor; each counts how far it rolls. The Pinpoint reads both pods and its built-in IMU, combines them about 1,500 times a second (goBILDA guide) and reports x, y and heading. On the board you can see the X and Y ports for the pods and an I²C port that goes to the robot's controller. goBILDA expects the robot to end within about 10 mm (0.5 in) of its start after being driven around and brought back.

Back to the demo: Every position number in the demo came from the Pinpoint. Point to where it sits on the robot. The PathLab notes say the Pinpoint localizer is assumed to work.

Ask the class: How many pods are there, and what does each one count? [Two; how far its wheel rolled.] Which part tells the Pinpoint which way the robot is facing? [The built-in IMU.]

Watch out: The photo is goBILDA's product image and shows a Pinpoint marked V2.0. The user guide I read is the 3110-0002-0001 (V1) guide, which gobilda.com also links from the V2 product page. The V2 page lists these differences: error checking on the I²C data, full 3D orientation (pitch, roll, quaternion), configurable bulk reads and a USB port for firmware updates. It gives the same 1,500 Hz update rate. I have not read a separate V2 guide, so check our unit's label before quoting guide details. The pods in the photo look like goBILDA's 4-Bar design, but I cannot confirm that from the image.

---

16 Which pod measures what

3 min.

Say: The **X pod** measures forward and back movement. The **Y pod** measures left and right movement (strafe). Each plugs into its own port. The robot's own axes: +X is forward and +Y is left (goBILDA guide). The names say what a pod **measures**, not where it sits.

Ask the class: Which pod rolls when the robot strafes? [The Y pod.] Which pod rolls when the robot drives straight ahead? [The X pod.]

Watch out: The robot picture is a generic example layout, not our robot. Each pod's wheel is drawn lying along the direction it rolls, with arrows. The offsets work the other way round: the guide says the X offset is measured sideways and the Y offset forward. Do not use the picture for offsets; those come from our Tuning Checklist.

Part C · How do I get there?

17 A path is a line or a curve between two Poses

3 min.

Say: A path is built from Poses, so a PoseFactory sits behind every path. The code names are `line` and `curve`.

Back to the demo: Test 1 used `Paths.line(startingPoint(), endpoint())`. Test 3 used `Paths.curve(startingPoint(), controlPoint1(), endpoint())` (PathLab walkthrough). The Pedro docs list `Paths.line`, `Paths.curve` and `Paths.path`.

Ask the class: What two things does a path need? [A start Pose and an end Pose; a curve also needs a control pose.]

Watch out: If you have a projector and wifi, open visualizer.pedropathing.com for a minute. Deck B teaches it properly.

---

18 A control pose pulls the path towards it

5 min.

Optional demo (skip if not possible): Hold a piece of string or a rubber band between two students and pull the middle with a finger.

Say: Think of a rubber band. A control pose in the middle bends the straight line towards it. More control poses give more bend.

Back to the demo: Live experiment, if time allows: ask for a prediction first, then change Control Point 1 on the Dashboard (Configuration → Path Poses), press INIT to see the new planned path, and run Test 3 again with A. Edits apply to the next run and are not saved when the robot powers off.

Ask the class: What happens with two control poses? [A taller, more complex bend.]

Watch out: No Bézier maths. “The control pose pulls the line towards it” is the right level. The docs call these Bézier curves.

---

19 Where you face is separate from where you go

6 min.

Optional demo (skip if not possible): One student walks along a curve four times: always facing forward, turning steadily, always facing the wall, always facing a cone.

Say: A car always faces where it drives. A mecanum robot can slide sideways while facing somewhere else, so we choose. Four modes from the Pedro docs: `tangent` (faces the way it moves, the default), `linear` (turns steadily), `constant` (one heading), `facingPoint` (always faces a point).

Back to the demo: This answers slide 10: Test 1 used `constant` (it kept the starting heading, 0°) and Test 3 used `tangent` (it faced along the path, so it ended facing about 72°). Next time try Test 2 (linear) and Test 7 (facing a point).

Ask the class: We must drive around an obstacle but keep the intake pointing at the goal the whole way. Which mode? [`facingPoint`, or `constant` if the goal direction never changes.]

Watch out: `piecewise` is shown only as "go deeper". Deck B covers it. PathLab also has Test 4 (`reverseTangent()`) and Test 6 (`Paths.through()`). I could not find those two calls in the Pedro 3 docs I read, only in the PathLab code. If a student asks, say "that is in our PathLab code, and I have not confirmed it in the Pedro docs." Do not teach them as Pedro features.

---

20 The follower drives. It updates every loop

6 min. The most important slide in the deck.

Say: Build the follower in `init()`, feed it in `loop()`, every loop. Write `follower.update();` on the whiteboard and leave it there.

Back to the demo: In the demo, pressing A starts the path with `follower.follow(path)`. The OpMode keeps calling `follower.update()` and waits while `follower.isBusy()`. After the End point it pauses 2 seconds and drives back. The OpMode is a state machine: READY, DRIVING_PATH, PAUSING_AT_ENDPOINT, RETURNING (and STOPPED on B). Deck B builds one.

Ask the class: What happens if I put a two-second wait inside loop()? [Pedro misses many updates, so its idea of where the robot is goes stale.]

Watch out: `setPose` must match where the robot is really placed at the start. This slide's code is from the Pedro docs (Setting Up Your Autonomous). Programming 101 knowledge of init() and loop() is assumed.

---

21 Foresight: a good driver brakes early

4 min.

Optional demo (skip if not possible): Have a student walk quickly towards a tape line and stop exactly on it, then do it again slowing down only at the last moment.
Natural break point after this slide.

Say: Foresight is Pedro 3's path-following algorithm. During tuning the robot is driven and braked to measure how far this robot slides. Pedro uses that to start slowing at the right moment. The two charts are illustrations, not measured data.

Back to the demo: Watch the end of the run again. Does it slow down before the End point? Does it stop on it or slide past? Report only what students actually see.

Ask the class: Why does Pedro need to measure our robot's braking? [Every robot slides differently.]

Watch out: Do not teach Foresight's numbers (kP, coefficients). That is for the tuning session with the robot, using our Tuning Checklist. **Check before saying the demo uses Foresight:** the PathLab notes say Foresight is configured in Constants, but the Constants sample in our Tuning Checker had Foresight commented out (`null`). Look at the real file first.

Part D · Putting it together

22 One loop. Every pass

4 min.

Say: Pinpoint measures, the localizer works out the pose, the follower compares it with your path and sets the wheel powers, and the motors move the robot. Then it all repeats.

Back to the demo: The Dashboard drew the plan (planned path) and the robot's position. The follower keeps trying to close the gap between them.

Ask the class: Cover the labels and have students rebuild the circle. Which box would you check if the robot is driving smoothly to the wrong place? [The Pinpoint and localizer.]

Watch out: The circle repeats as fast as the program loops. Do not give a number of times per second.

---

23 Now answer your own questions

6 min. Uses the board from slide 3.

Say: Go down the board. For each question, find its row in the table, say the idea in one sentence, and tick it off. Questions that need code go to Deck B. Then run the demo once more.

Back to the demo: Check each earlier “hold that thought”: the 90° End-point heading (slide 10) and “off the path according to whom?” (slide 14).

Ask the class: What do you notice now that you missed the first time? [Likely: heading, the bend, the slowing, the pause.]

Watch out: Do not force a link. If a question is not on the table, say “good question” and put it on a list for the next session. The last row (“Why did it come back by itself?”) is about our OpMode's state machine, which is code, so it belongs to Deck B.

---

24 Fifteen words you now own

3 min.

Say: Cover the right-hand column and quiz. The new word since the first draft is IMU.

---

25 Six questions

7 min.

Say: Let students answer out loud before you reveal anything.

Ask the class:

1. Why does “2 seconds at 50%” vary? [Battery, slipping wheels, bumps or a different floor. Timing measures time, not position.]
2. Robot turns 90° on the spot: what changes? [Only the heading.]
3. What does a mirrored PoseFactory save? [Redrawing every Pose for the other alliance. We still check headings in the Visualizer.]
4. What goes wrong with odometry on a long walk? [Small errors add up: drift.]
5. Why update every loop? [The follower needs a fresh pose each time to steer. Skip it and it acts on old information.]
6. Test 1 and Test 3 ended at the same spot. What was different? [Test 1 was a straight line that kept its heading (constant), so it ended facing 0°. Test 3 bent towards a control point and faced along the path (tangent), so it ended facing about 72°.]

Watch out: Book the practical session (Deck B) before everyone leaves the room.

Sources used in this deck

• Pedro Pathing docs: pedropathing.com/docs (Introduction, Creating a Pose, Pose Factory, Creating a Path, Interpolation, Bézier Curves, Coordinates, Setting Up Your Autonomous, TeleOp Usage, Tuning).

• goBILDA Pinpoint Odometry Computer User Guide (3110-0002-0001): 1,500 Hz update rate, built-in IMU and its power-up calibration, about 10 mm return-to-start accuracy, X pod = forward, Y pod = sideways, +X forward and +Y left, and the 4-Bar pod figures (32 mm wheel, 2,000 counts

per turn). The 4-Bar Odometry Pod page on gobilda.com agrees on those two figures. Note that 3110-0002-0001 is marked discontinued on gobilda.com; a Pinpoint V2 (3110-0002-0002) is the current model.

• PathLab (github.com/smrc-pune/PathLab): Path Tests User Guide, HLD and LLD. Default points, test descriptions, expected distances and end headings, state machine, Dashboard address, troubleshooting. Source code not read.

• Sripada and Henderson, the Pedro Pathing paper: the braking-distance idea (footnote 2), as summarised in our Constants guide.

• Pedro Visualizer (visualizer.pedropathing.com): field size 141.5 in; Java export uses PoseFactory.

• Photos on slides 14 and 15: goBILDA product images (gobilda.com), supplied by the team.

General reasoning and physics (slide 5 on timing, the IMU explanation on slide 11, and the distance = turns × rim length idea on slide 14): not claims from the sources above. The box on slide 8 is our own simplification.